Эта страница была переведена с английского языка силами сообщества. Вы тоже можете внести свой вклад, присоединившись к русскоязычному сообществу MDN Web Docs.

Операторы сравнения

JavaScript предоставляет три оператора сравнения величин:

- равенство ("двойное равно") использует `==` (en-US),
- строгое равенство ("тройное равно" или "идентично") использует `===` (en-US),
- и `Object.is` (новшество из ECMAScript 6).

Выбор оператора зависит от типа сравнения, которое необходимо произвести.

В общих чертах, двойное равно перед сравнением величин производит приведение типов; тройное равно сравнивает величины без приведения (если величины разных типов, вернёт `false`, даже не сравнивая); ну и `Object.is` ведёт себя так же, как и тройное равно, но со специальной обработкой для `NaN`, `-0` и `+0`, возвращая `false` при сравнении `-0` и `+0`, и `true` для операции `Object.is(NaN, NaN)`. (В то время как двойное или тройное равенство вернут `false` согласно стандарту IEEE 754.) Следует отметить, что все эти различия в сравнениях применимы лишь для примитивов. Для любых не примитивных объектов `x` и `y`, которые имеют одинаковые структуры, но представляют собой два отдельных объекта (переменные `x` и `y` не ссылаются на один и тот же объект), все операторы сравнения вернут `false`.

Сравнение с использованием `==`

Перед сравнением оператор равенства *приводит* обе величины к общему типу. После приведений (одного или обоих операндов), конечное сравнение выполняется также как и для `===`. Операция сравнения *симметрична*: `A == B` возвращает то же значение, что и `B == A` для любых значений `A` и `B`.

В следующей таблице приведены результаты сравнения оператора равенства для различных значений:

	Операнд B						
		Undefined	**Null**	**Number**	**String**	**Boolean**	**Object**

		Операнд B					
Операнд A	Undefined	true	true	false	false	false	false
	Null	true	true	false	false	false	false
	Number	false	false	A === B	A === ToNumber(B)	A === ToNumber(B)	A == ToPrimitive(B)
	String	false	false	ToNumber(A) === B	A === B	ToNumber(A) === ToNumber(B)	A == ToPrimitive(B)
	Boolean	false	false	ToNumber(A) === B	ToNumber(A) === ToNumber(B)	A === B	ToNumber(A) == ToPrimitive(B)
	Object	false	false	ToPrimitive(A) == B	ToPrimitive(A) == B	ToPrimitive(A) == ToNumber(B)	A === B

В таблице выше, ToNumber(A) пытается перед сравнением привести свой аргумент к числу. Такое поведение эквивалентно `+A` (унарный оператор `+`). Если ToPrimitive(A) получает *объект* в качестве аргумента, то производятся попытки привести его к примитиву, вызывая на нем методы `A.toString` и `A.valueOf`.

Традиционно (и в соответствии с ECMAScript), ни один из объектов не равен `undefined` или `null`. Но большинство браузеров позволяет определённому классу объектов (в частности, объектам `document.all` для любой страницы) *эмулировать* значение `undefined`. Оператор равенства вернёт значение true для `null == A` и `undefined == A`, тогда, и только тогда, когда объект эмулирует значение `undefined`. Во всех остальных случаях объект не может быть равен `undefined` или `null`.

```
var num = 0;
var obj = new String("0");
var str = "0";
var b = false;

console.log(num == num); // true
console.log(obj == obj); // true
console.log(str == str); // true

console.log(num == obj); // true
console.log(num == str); // true
```

```
console.log(obj == str); // true
console.log(null == undefined); // true

// оба false, кроме очень редких случаев
console.log(obj == null);
console.log(obj == undefined);
```

Некоторые разработчики считают, что лучше всегда употреблять оператор *строго* равенства, вместо сравнения с приведением типов. Результат строгого равенства легче предугадать, да и сравнивая значения, без их приведения, можно получить выигрыш в скорости .

Строгое равенство с использованием `===`

Строгое равно проверяет на равенство две величины, при этом тип каждой из величин перед сравнением не изменяется (не приводится). Если значения имеют различающиеся типы, то они не могут быть равными. С другой стороны все не числовые переменные, принадлежащие одному типу, считаются равными между собой, если содержат одинаковые величины. Ну и, наконец, числовые переменные считаются равными, если они имеют одинаковые значения, либо одна из них `+0`, а вторая `-0`. В то же время, если хотя бы одна из числовых переменных содержит значение `NaN`, выражение вернёт `false`.

```
var num = 0;
var obj = new String("0");
```

```
var str = "0";
var b = false;

console.log(num === num); // true
console.log(obj === obj); // true
console.log(str === str); // true

console.log(num === obj); // false
console.log(num === str); // false
console.log(obj === str); // false
console.log(null === undefined); // false
console.log(obj === null); // false
console.log(obj === undefined); // false
```

Практически всегда для сравнения следует использовать оператор строгого равенства. Для всех значений, за исключением числовых используется очевидная семантика: величина равна только сама себе. Как было сказано выше для числовых типов можно выделить два особых случая. Во-первых, сравнение `+0` и `-0`. Знак для нуля введён для упрощения некоторых вычислений с плавающей запятой, однако, с точки зрения математики, разницы между `+0` и `-0` не существует, поэтому оператор строгого равенства считает их равными. Во-вторых, сравнение величин `NaN`. `NaN` (Not a number) представляет из себя значение не определённой величины, которое применяется для не чётко определённых математических задач (например `+∞ + -∞`). Для оператора строго равенства `NaN` не равен ни одной из величин, в том числе и самому себе (единственный случай, когда `(x!==x)` вернёт true).

Равенство одинаковых величин

Равенство одинаковых величин определяет *функциональную идентичность* во всех контекстах сравниваемых величин. (Данный способ сравнения основывается на [принципе подстановки Барбары Лисков](#) .) Рассмотрим пример попытки изменения неизменяемого (immutable) свойства :

```
// Добавление неизменяемого свойства NEGATIVE_ZERO (отрицательный ноль) в конструктор Number.
Object.defineProperty(Number, "NEGATIVE_ZERO", {
  value: -0,
  writable: false,
  configurable: false,
  enumerable: false,
});

function attemptMutation(v) {
  Object.defineProperty(Number, "NEGATIVE_ZERO", { value: v });
}
```

При попытке изменения неизменяемого свойства, вызов `Object.defineProperty` выбросит исключение, однако, если новое свойство равняется старому, изменений не произойдёт и исключение не будет выброшено. Если `v` содержит `-0`, изменений не произойдёт, а значит, код отработает без выброса

исключений. Однако, если же `v` содержит `+0`, `Number.NEGATIVE_ZERO` утратит свою неизменяемую величину.

Именно для сравнения нового и текущего неизменяемых свойств используется сравнение одинаковых величин, представленное методом `Object.is`.

Равенство одинаковых величин и нулей

Аналогично равенству одинаковых величин, но +0 и -0 считаются равными.

Равенство одинаковых величин и нулей не предоставляется как JavaScript API, но может быть реализовано с помощью пользовательского кода:

```
function sameValueZero(x, y) {
  if (typeof x === "number" && typeof y === "number") {
    // x и y равны (могут быть -0 и 0) или они оба равны NaN
    return x === y || (x !== x && y !== y);
  }
  return x === y;
}
```

Равенство одинаковых величин и нулей отличается от *строгого равенства* тем, что принимает каждое

значение `NaN` равным любому другому значению `NaN`, а от *равенства одинаковых величин* тем, что принимает `-0` равным `0`. Подобное поведение обычно оказывается самым уместным при поиске в списках, особенно при работе с `NaN`. Данная стратегия сравнения используется в методах `Array.prototype.includes()`, `TypedArray.prototype.includes()` (en-US), а так же в `Map` (en-US) и `Set` для оценки равенства ключей.

Спецификации для равенства, строгого равенства и равенства одинаковых величин

В стандарте ES5, сравнение выполняемое оператором `==` (en-US) описывается в секции 11.9.3, The Abstract Equality Algorithm. Описание оператора `===` (en-US) находится в секции 11.9.6, The Strict Equality Algorithm. В секции 9.12, The SameValue Algorithm ES5 описывает операцию сравнение одинаковых величин для внутреннего движка JS. Строгое равенство и равенство одинаковых величин, практически одинаковы, за исключением обработки числовых типов. ES6 предлагает использовать алгоритм сравнения одинаковых величин через вызов `Object.is`.

Как понимать все эти способы сравнения?

До выхода редакции ES6 считалось, что оператор строгого равенства просто "улучшенная" версия оператора нестрогого равенства. Например, некоторые считали, что == просто улучшенная версия === потому, что первый оператор делает всё то же, что и второй, плюс приведение типов своих операндов. То

есть 6 == "6". (Или же наоборот: оператор нестрогого равенства базовый, а оператор строгого равенства просто его улучшенная версия, ведь он добавляет ещё одно условие - требует, чтобы оба операнда были одного и того же типа. Какой вариант ближе вам, зависит только от вашей точки зрения на вещи.)

Но эти точки зрения уже нельзя применить к новому методу сравнения Object.is из новой редакции ES6. Нельзя сказать, что Object.is более или менее строг относительно существующих способов сравнения, или что это нечто среднее между ними. Ниже в таблице показаны основные различия операторов сравнения. Object.is интересен тем, что различает -0 и +0, а также умеет сравнивать два не числа NaN.

x	y	==	===	Object.is	SameValueZero
undefined	undefined	✅ true	✅ true	✅ true	✅ true
null	null	✅ true	✅ true	✅ true	✅ true
true	true	✅ true	✅ true	✅ true	✅ true
false	false	✅ true	✅ true	✅ true	✅ true
'foo'	'foo'	✅ true	✅ true	✅ true	✅ true
0	0	✅ true	✅ true	✅ true	✅ true

x	y	==	===	Object.is	SameValueZero
+0	-0	✅ true	✅ true	❌ false	✅ true
+0	0	✅ true	✅ true	✅ true	✅ true
-0	0	✅ true	✅ true	❌ false	✅ true
0n	-0n	✅ true	✅ true	✅ true	✅ true
0	false	✅ true	❌ false	❌ false	❌ false
""	false	✅ true	❌ false	❌ false	❌ false
""	0	✅ true	❌ false	❌ false	❌ false
'0'	0	✅ true	❌ false	❌ false	❌ false
'17'	17	✅ true	❌ false	❌ false	❌ false
[1, 2]	'1,2'	✅ true	❌ false	❌ false	❌ false
new String('foo')	'foo'	✅ true	❌ false	❌ false	❌ false
null	undefined	✅ true	❌ false	❌ false	❌ false

x	y	==	===	Object.is	SameValueZero
null	false	❌ false	❌ false	❌ false	❌ false
undefined	false	❌ false	❌ false	❌ false	❌ false
{ foo: 'bar' }	{ foo: 'bar' }	❌ false	❌ false	❌ false	❌ false
new String('foo')	new String('foo')	❌ false	❌ false	❌ false	❌ false
0	null	❌ false	❌ false	❌ false	❌ false
0	NaN	❌ false	❌ false	❌ false	❌ false
'foo'	NaN	❌ false	❌ false	❌ false	❌ false
NaN	NaN	❌ false	❌ false	✅ true	✅ true

Когда использовать Object.is(), а когда ===

Особое поведение `Object.is` по отношению к нулям, вероятно, будет представлять интерес при реализации определенных схем метапрограммирования, особенно в отношении дескрипторов свойств, когда желательно, чтобы ваш алгоритм имел такое же поведение, как `Object.defineProperty`. В случае, если вам этого не требуется, рекомендуется избегать `Object.is` и вместо этого использовать `===`. Как правило, даже

при необходимости сравнения `NaN`-значений, эффективнее выполнить проверку таких значений отдельно, при помощи метода `isNaN`, чем выяснять, как окружающие вычисления могут повлиять на знаки нулей и как это отразится на вашем алгоритме.

Вот неполный список встроенных методов и операторов, при использовании которых, различие между `-0` и `+0` может оказаться важным для вашего кода:

`-` (unary negation)

Рассмотрим следующий пример:

```
const stoppingForce = obj.mass * -obj.velocity;
```

Если `obj.velocity` равно `0` (или вычисляется как `0`), в этом месте возникает `-0` и присваивается в `stoppingForce`.

`Math.atan2`, `Math.ceil`, `Math.pow`, `Math.round`

In some cases, it's possible for a `-0` to be introduced into an expression as a return value of these methods even when no `-0` exists as one of the parameters. For example, using `Math.pow` to raise `-Infinity` to the power of any negative, odd exponent evaluates to `-0`. Refer to the documentation for the individual methods.

`Math.floor`, `Math.max`, `Math.min`, `Math.sin`, `Math.sqrt`, `Math.tan`

It's possible to get a `-0` return value out of these methods in some cases where a `-0` exists as one of the parameters. E.g., `Math.min(-0, +0)` evaluates to `-0`. Refer to the documentation for the individual methods.

`~`, `<<`, `>>`

Each of these operators uses the ToInt32 algorithm internally. Since there is only one representation for 0 in the internal 32-bit integer type, `-0` will not survive a round trip after an inverse operation. E.g., both `Object.is(~~(-0), -0)` and `Object.is(-0 << 2 >> 2, -0)` evaluate to `false`.

Relying on `Object.is` when the signedness of zeros is not taken into account can be hazardous. Of course, when the intent is to distinguish between `-0` and `+0`, it does exactly what's desired.

Caveat: Object.is() and NaN

The `Object.is` specification treats all instances of `NaN` as the same object. However, since typed arrays are available, we can have distinct floating point representations of `NaN` which don't behave identically in all contexts. For example:

```
const f2b = (x) => new Uint8Array(new Float64Array([x]).buffer);
const b2f = (x) => new Float64Array(x.buffer)[0];
```

```
// Get a byte representation of NaN
const n = f2b(NaN);
// Change the first bit, which is the sign bit and doesn't matter for NaN
n[0] = 1;
const nan2 = b2f(n);
console.log(nan2); // NaN
console.log(Object.is(nan2, NaN)); // true
console.log(f2b(NaN)); // Uint8Array(8) [0, 0, 0, 0, 0, 0, 248, 127]
console.log(f2b(nan2)); // Uint8Array(8) [1, 0, 0, 0, 0, 0, 248, 127]
```

Смотрите также

- JS Comparison Table

This page was last modified on 2 мар. 2023 г. by MDN contributors.